

Lab 2: Vibe Code the Northstar Studio Website

Course: AI Agents with Gemini Spark (TGS-2026064173)

Learning objective: Create and test a responsive website through a bounded vibe-coding loop using Claude Code, Codex or Antigravity.

Assessment alignment: A2, K4, K5

Tools: Claude Code, Codex or Antigravity; Git; HTML; CSS; JavaScript; browser testing

Scenario

Northstar Studio is a small Singapore design business preparing a responsive website and a governed content workflow. You are the human owner of the outcome. Gemini Spark may research, organise or draft within the approved boundary; Claude Code, Codex or Antigravity may edit the local website repository. Publishing, sending, purchases, credential entry and sensitive-data changes always require human control.

Deliverable

A working one-page HTML/CSS/JavaScript website, responsive at three widths, with accessible navigation, an evidence log and a rollback commit.

Before you start

1. Work only with the supplied mock data and project folder.
2. Never paste passwords, payment details, live customer data or API keys into an agent task.
3. Read the requested plan and inspect every file diff before accepting a change.
4. Stop if the agent requests a new tool, site, permission or consequential action outside the approved contract.

Detailed procedure

Step 1 - Copy starter/ to working-site/ and open the folder in your chosen coding agent.

Work with: `cp -R starter working-site`

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 2 - Initialize version control and create a baseline commit before the agent edits files.

Work with: `cd working-site && git init && git add index.html styles.css script.js && git commit -m 'chore: baseline starter'`

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 3 - Give the agent the design brief, acceptance checklist and guardrails; ask for a plan only.

Read `../design-brief.md` and `../acceptance-checklist.md`. Propose a five-step plan. Do not edit files yet.

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 4 - Review the plan and approve only the first thin slice: semantic page structure and navigation.

Implement only step 1. Preserve existing filenames and use no external framework.

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 5 - Inspect the diff and reject unrelated changes before opening the site.

Work with: `git diff -- index.html styles.css script.js`

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 6 - Run a local server and open the site in a browser.

Work with: `python3 -m http.server 8000`

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 7 - Ask for the visual system: spacing scale, type scale, color tokens and reusable card/button styles.

Implement the visual system only. Keep contrast at WCAG AA and explain the tokens you add.

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 8 - Review the diff and test at 1440 px, 768 px and 390 px widths.

Work with: Open <http://localhost:8000> and use responsive device mode.

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 9 - Ask for the contact form interaction with client-side validation and clear success/error states.

Add accessible client-side validation. Do not send data or add a backend.

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 10 - Use keyboard-only navigation to test the menu, links and form controls.

Work with: Tab through every interactive element; verify a visible focus indicator.

Review before continuing

- Confirm the result serves the stated objective.

- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 11 - Run the supplied checks and record results in evidence/test-results.md.

Work with: `python3 ../scripts/check_site.py working-site`

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 12 - Ask the agent to fix only failed checks; re-run tests after each small change.

Fix only the failures listed in `../evidence/test-results.md`. Explain each change.

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 13 - Review all claims against content-inventory.csv and remove invented facts.

Work with: `content-inventory.csv`

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Step 14 - Commit the approved website and record the commit hash as the rollback point.

Work with: `git add index.html styles.css script.js && git commit -m 'feat: build reviewed responsive site'`

Review before continuing

- Confirm the result serves the stated objective.
- Confirm only approved files, data and tools were used.
- Capture the named evidence artifact or note why the step stopped.

Acceptance test

The website renders without errors, passes the supplied structural checks, works at desktop/tablet/mobile widths, is keyboard navigable, contains only approved claims, and has an identifiable rollback commit.

Evidence checklist

- All required working files are present.
- The permission boundary was followed.
- Human approval is recorded at every required gate.
- The acceptance test was run and the result captured.
- Any failure or limitation is recorded honestly.
- No secret, live personal data or answer key is present.

Clean-up / rollback

Pause any schedule created for the lab, stop unfinished tasks, disconnect unnecessary apps, and restore the recorded baseline or rollback commit if the final evidence does not pass.